

Lecture 8: SQL

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Assignments

- PA1 “Run All” issue regraded
 - If you click “Run All,” it will stop at the first error
- WA2, PA2 due Sunday

Important tools

- **Python:** A user-friendly language for data analysis.
- **Git:** Helps manage code and data changes when working with a team.
- **Pandas:** Simplifies data cleaning and manipulation.
- **Databases:** Needed for storing and retrieving data efficiently.

Knowing how to use these is essential for effective data analysis, collaboration, and handling data in real-world projects.

Next

Databases & SQL

Databases

Many tasks in data science involve extracting and manipulating data from a database. Most leading database systems are relational, organizing data in rows and columns of tables.

- Databases are used when:
 - Dealing with large amounts of data
 - Our data won't fit in memory
 - We want to access particular parts of our data extremely quickly

Databases

For example, imagine the table on the right contains ten million customers, and we would like to grab every customer in Berlin quickly.

- Allows efficient storage and management of the data on disk.
- Allows retrieval of relevant data without the need to load the entire dataset into memory.

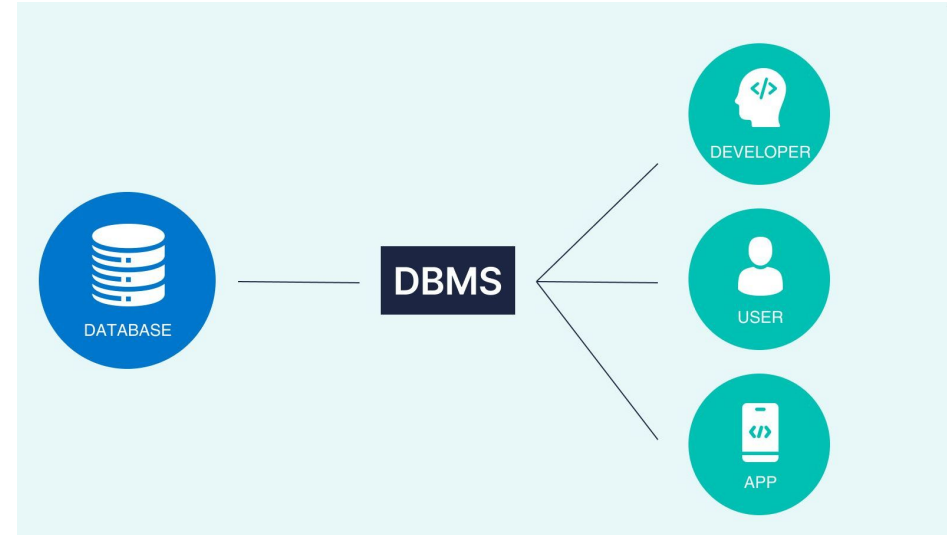
Number of Records: 91

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden
6	Blauer See Delikatessen	Hanna Moos	Forsterstr. 57	Mannheim	68306	Germany
7	Blondel père et fils	Frédérique Citeaux	24, place Kléber	Strasbourg	67000	France
8	Bóldo Comidas preparadas	Martín Sommer	C/ Araquil, 67	Madrid	28023	Spain
9	Bon app'	Laurence Lebihans	12, rue des Bouchers	Marseille	13008	France
10	Bottom-Dollar Marketse	Elizabeth Lincoln	23 Tsawassen Blvd.	Tsawassen	T2F 8M4	Canada
11	B's Beverages	Victoria Ashworth	Fauntleroy Circus	London	EC2 5NT	UK
12	Cactus Comidas para llevar	Patricio Simpson	Cerrito 333	Buenos Aires	1010	Argentina
13	Centro comercial Moctezuma	Francisco Chang	Sierras de Granada 9993	México D.F.	05022	Mexico
14	Chop-suey Chinese	Yang Wang	Hauptstr. 29	Bern	3012	Switzerland
15	Comércio Mineiro	Pedro Afonso	Av. dos Lusíadas, 23	São Paulo	05432-043	Brazil

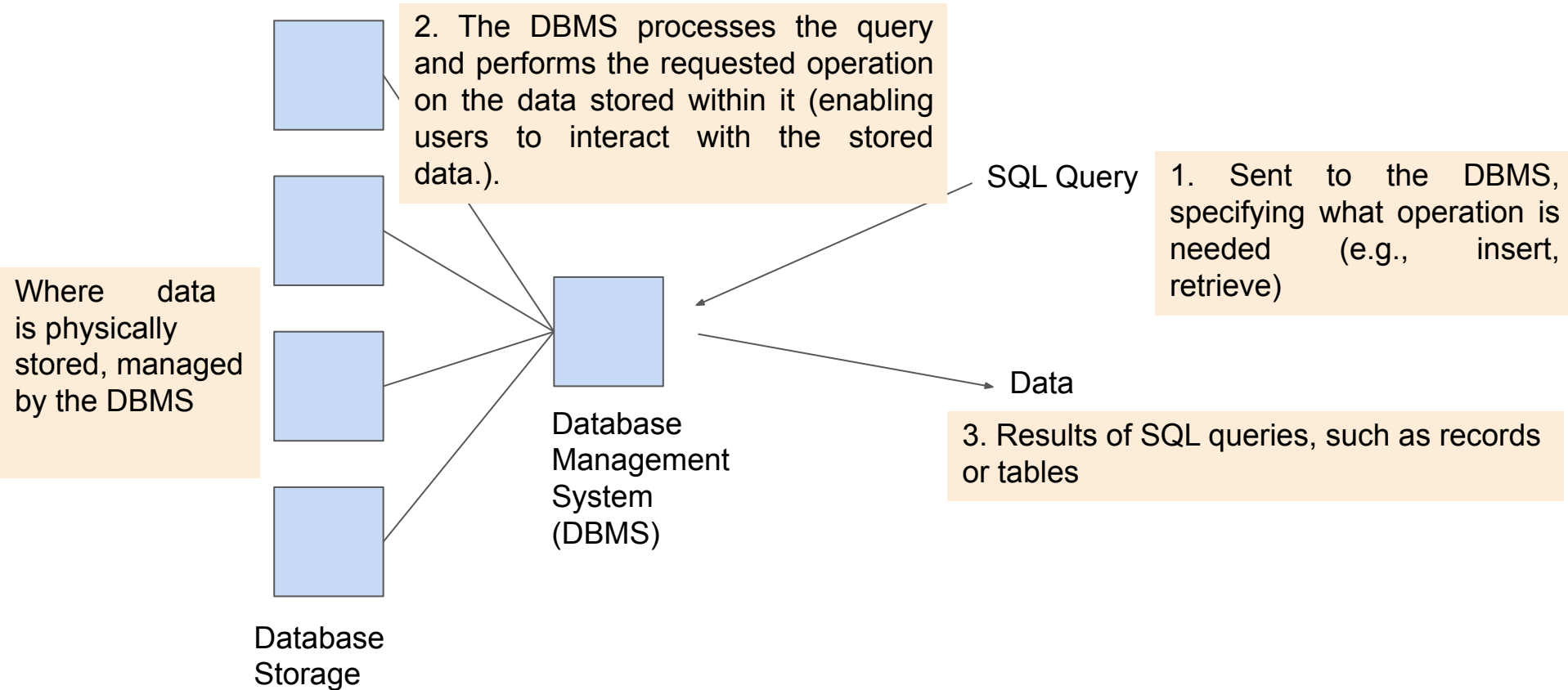
DBMS (Database Management System)

A DBMS (Database Management System) is software that manages and controls access to databases.

- Provides a systematic way to store, retrieve, and manage data, enabling users and applications to interact with the data efficiently.

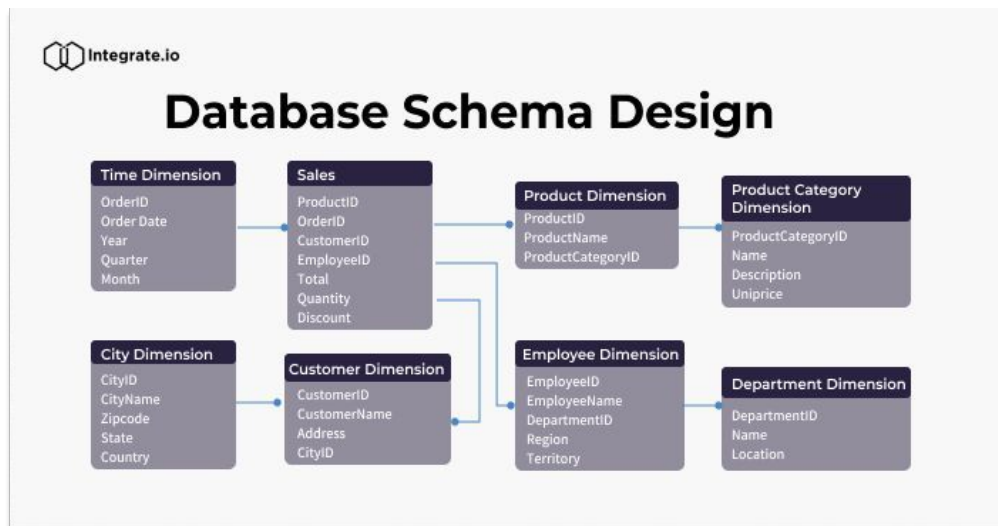


Database Setup (Very Simplified)



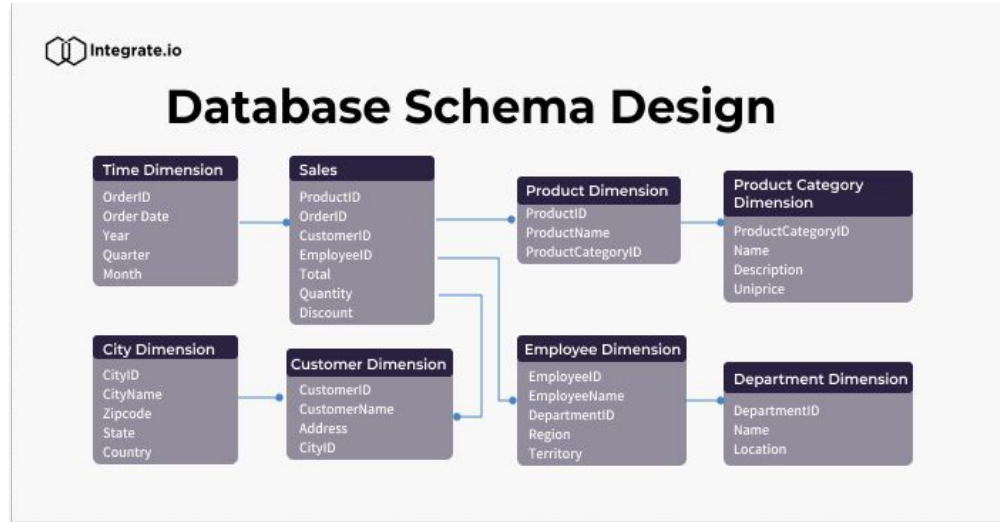
Important Vocab

- Databases store data in **tables**, which are similar to DataFrames in Pandas.
 - Organized into rows and columns
 - Has an **index**, enable quicker data retrieval and faster querying
- A **database schema** defines the **structure of the database**:
 - including tables, their columns, data types, relationships, and constraints
 - outlines how the data is organized



Important Vocab

- Two tables may often have the same field despite having different information
 - E.g, a 'customer' table and an 'order' table may both contain a 'customer_id' field to establish a relationship between them.



SQL

SQL is a computer language for manipulating and retrieving data. It is the standard language for relational databases and is also supported by many non-relational databases. SQL is pronounced either "S-Q-L " or "seekwəl ".

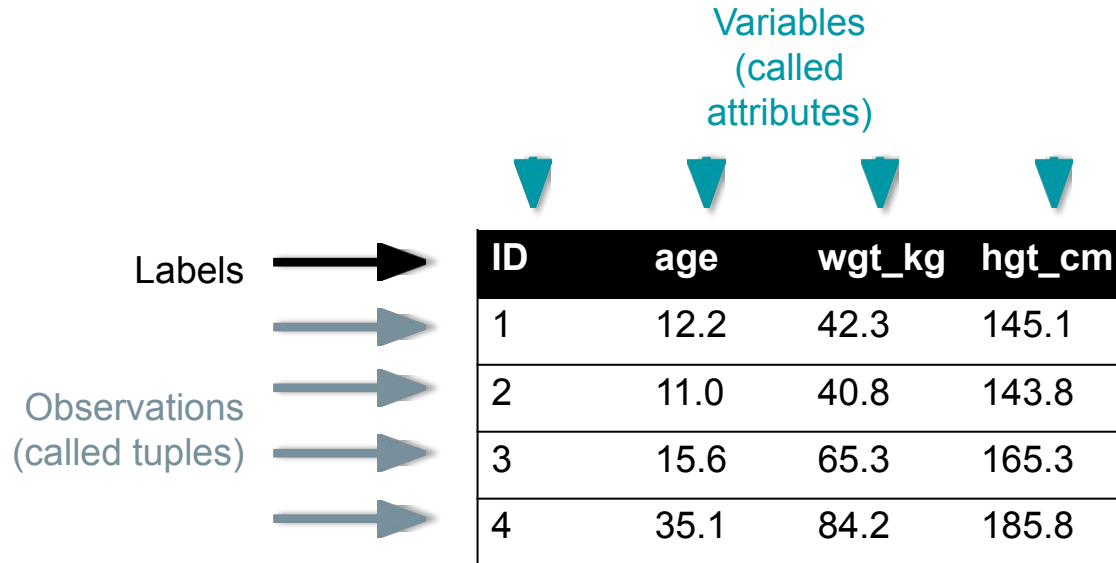
- SQL is how we ask our database questions
- Stands for **Structured Query Language**
- It's not a programming language
- For example, if we want to ask for all customers from Berlin:

```
SELECT * FROM customers WHERE City = 'Berlin'
```

RELATION

Simplest relation: a table aka tabular data full of unique tuples

Variables
(called
attributes)



ID	age	wgt_kg	hgt_cm
1	12.2	42.3	145.1
2	11.0	40.8	143.8
3	15.6	65.3	165.3
4	35.1	84.2	185.8

PRIMARY KEYS

ID	age	wgt_kg	hgt_cm	nat_id
1	12.2	42.3	145.1	1
2	11.0	40.8	143.8	1
3	15.6	65.3	165.3	2
4	35.1	84.2	185.8	1
5	18.1	62.2	176.2	3
6	19.6	82.1	180.1	1

ID	Nationality
1	USA
2	Canada
3	Mexico

The primary key is a unique identifier for every tuple in a relation

- Each tuple has exactly one primary key

FOREIGN KEYS

ID	age	wgt_kg	hgt_cm	nat_id
1	12.2	42.3	145.1	1
2	11.0	40.8	143.8	1
3	15.6	65.3	165.3	2
4	35.1	84.2	185.8	1
5	18.1	62.2	176.2	3
6	19.6	82.1	180.1	1

ID	Nationality
1	USA
2	Canada
3	Mexico

Foreign keys are attributes (columns) that point to a different table's primary key

- A table can have **multiple foreign keys**.

RELATION SCHEMA

A relational schema is a blueprint that defines the structure of a relational database, including its tables, columns, data types, and relationships between tables.

A list of all the attribute names, and their *domains*

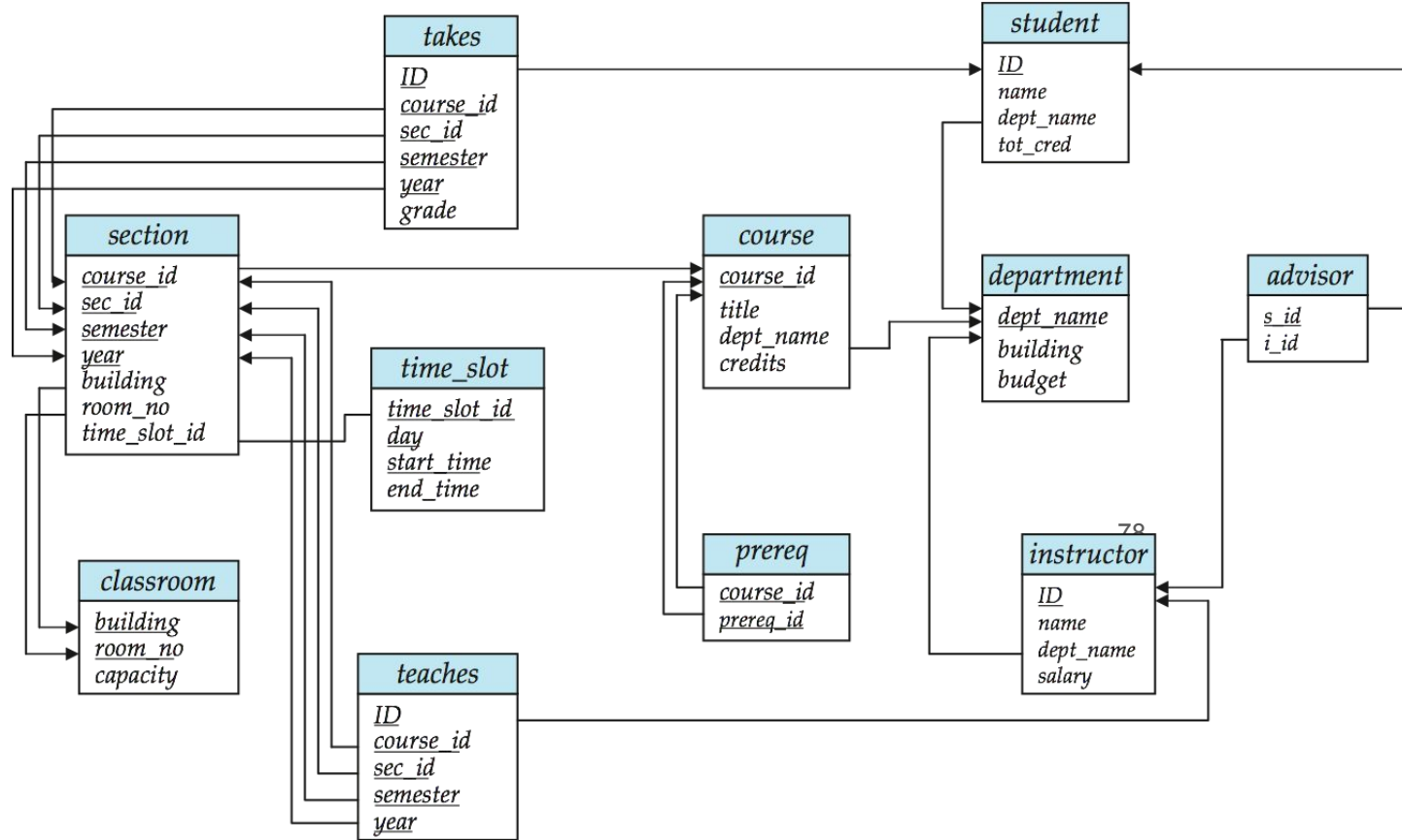
```
create table department
(dept_name
varchar(20), building
varchar(15),
budget numeric(12,2) check (budget >
0),
primary key (dept_name)
);
```

SQL Statements

To create
Tables

```
create table instructor (
  ID   char(5),
  name  varchar(20) not null,
  dept_name varchar(20),
  salary numeric(8,2),
  primary key (ID),
  foreign key (dept_name) references department
);
```

SCHEMA DIAGRAMS



Basic SQL Command: Example

CREATE TABLE: Create a new table in the database.

```
CREATE TABLE Customers (  
  CustomerID INT PRIMARY KEY  
  AUTO_INCREMENT, Name VARCHAR(50),  
  Email VARCHAR(100) );
```

INSERT INTO: Adds new rows of data to a table.

```
INSERT INTO Customers (CustomerID, Name, Email)  
VALUES (1, 'John Doe', 'john.doe@example.com');
```

SELECT: Retrieves data from one or more tables.

```
SELECT Name, Email FROM Customers
```

WHERE: Filters records based on a condition.

```
SELECT Name, Email FROM Customers WHERE  
CustomerID = 1;
```

UPDATE Modifies existing data in a table.

```
UPDATE Customers  
SET Email = 'john.doe@newdomain.com'  
WHERE CustomerID = 1;
```

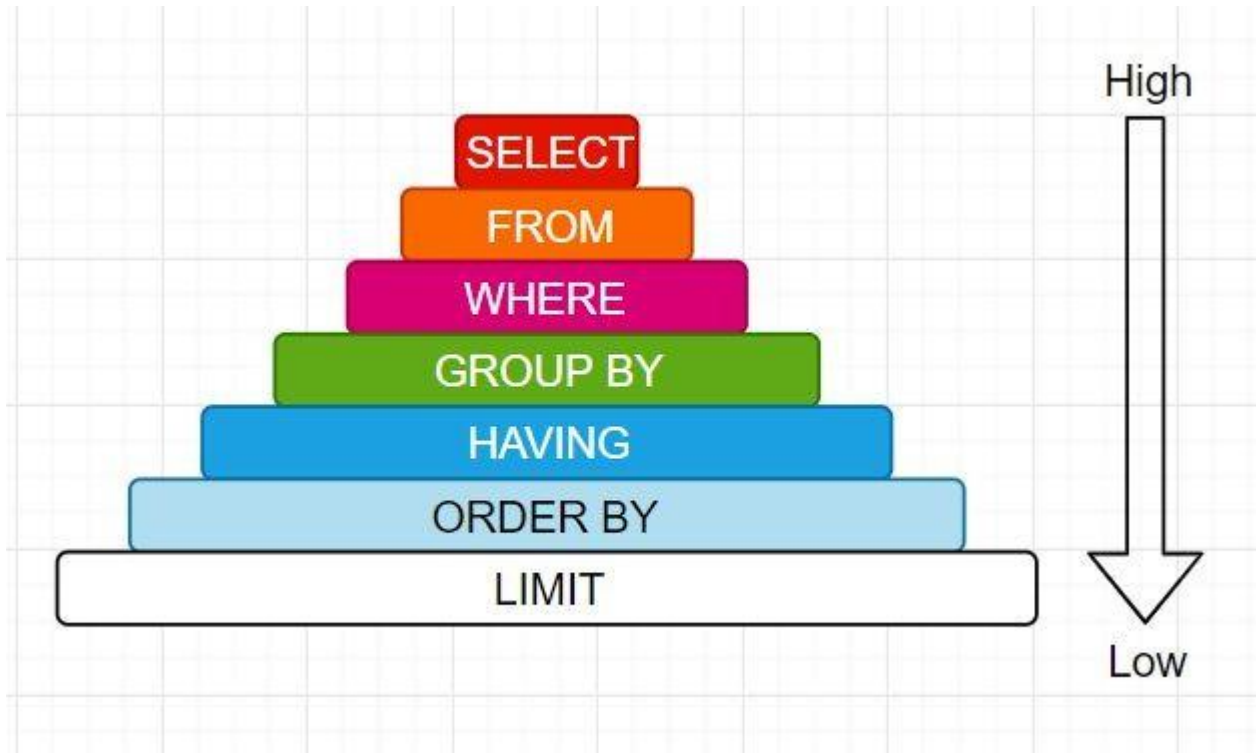
DELETE: Removes rows from a table.

```
DELETE FROM Customers WHERE CustomerID = 1;
```

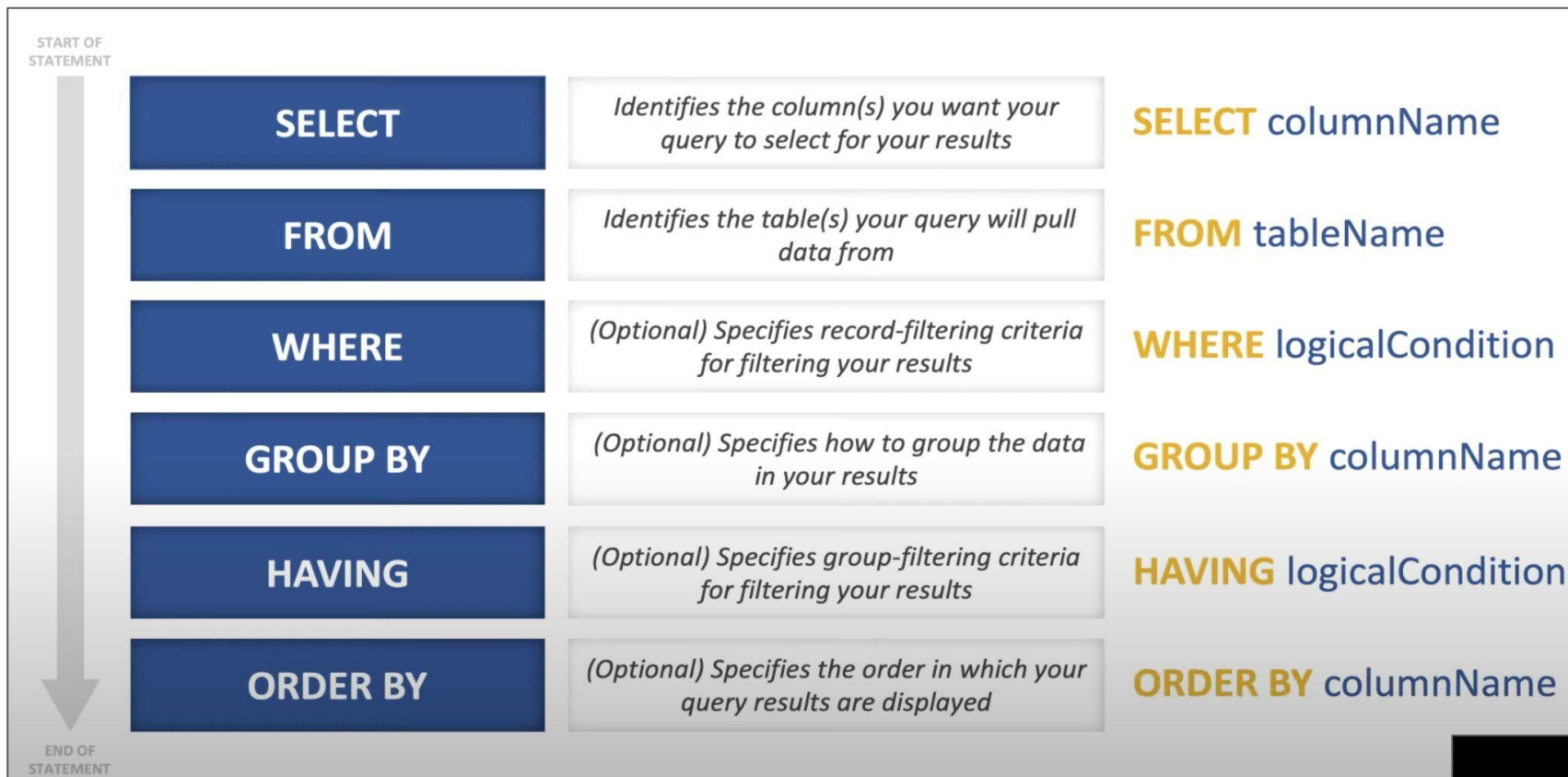
DROP TABLE: Delete an entire table from the database, including all its data and structure.

```
DROP TABLE Customers;
```

The “Big 6” Elements of a SQL Select Statement



The “Big 6” Elements of a SQL Select Statement



How a query is structured (Where condition)

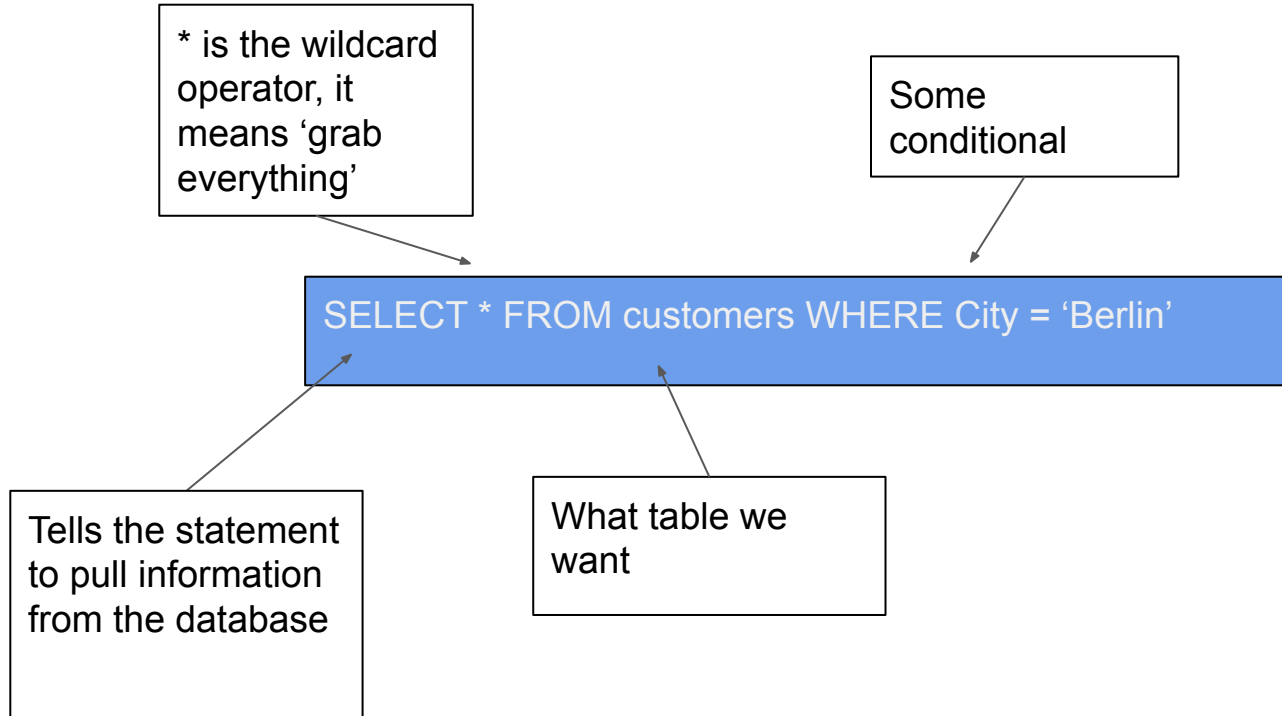
* is the wildcard operator, it means 'grab everything'

Some conditional

```
SELECT * FROM customers WHERE City = 'Berlin'
```

Tells the statement to pull information from the database

What table we want



2. How a SELECT query is structured (ORDER BY condition)

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

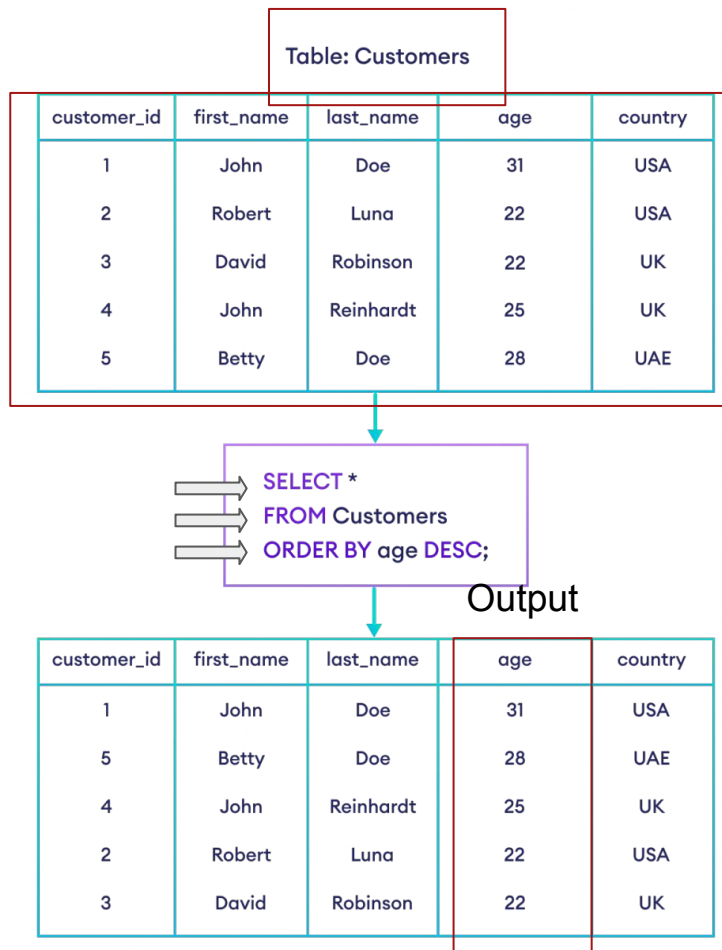
SELECT *
FROM Customers
ORDER BY age ASC;

Try DESC!

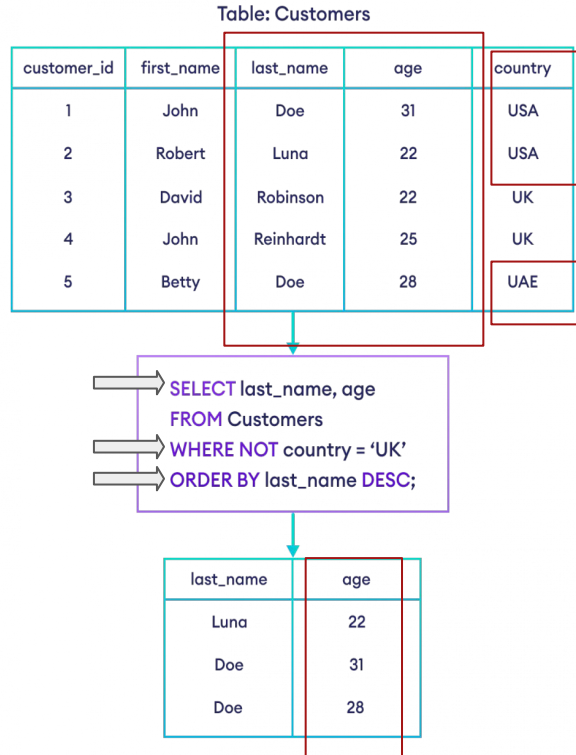
Output

customer_id	first_name	last_name	age	country
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE
1	John	Doe	31	USA

3. How a SELECT query is structured (ORDER BY DESC condition)



4. How a SELECT query is structured (**WHERE** and **ORDER BY** condition)



5. How a SELECT query is structured (Combining Conditions with AND, OR, and NOT condition)

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Return rows where the Country is USA and the Age is greater than 30;

```
SELECT first_name, email FROM Customers  
WHERE country = 'USA' AND age > 30;
```


6. How a SELECT query is structured (DISTINCT to Remove Duplicates)

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Return a list of unique countries from the Customers table.

```
SELECT DISTINCT Country FROM Customers;
```

7. How a SELECT query is structured for **aggregate** function (e.g. **SUM**)

Table: Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

`SELECT SUM(amount) AS total_sales
FROM Orders;`

total_sales
13350

column new name "total_sales"

```
SELECT SUM(column_name)  
FROM table;
```

8. How a SELECT query is structured **Aggregating Data** with **GROUP BY**

The GROUP BY clause **groups rows** that have the same values in specified columns. Commonly used with aggregate functions like COUNT(), SUM(), AVG(), MIN(), and MAX() to ***perform operations on each group of data.***

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

- **column_name:** The column you want to group by.
- **aggregate_function(column_name):** An aggregate function applied to each group. For example, COUNT(), SUM(), etc.
- **table_name:** The name of the table from which the data is being selected.

Table: sales

employee_id	sales_amount
1	500
2	300
1	400
2	700
3	600

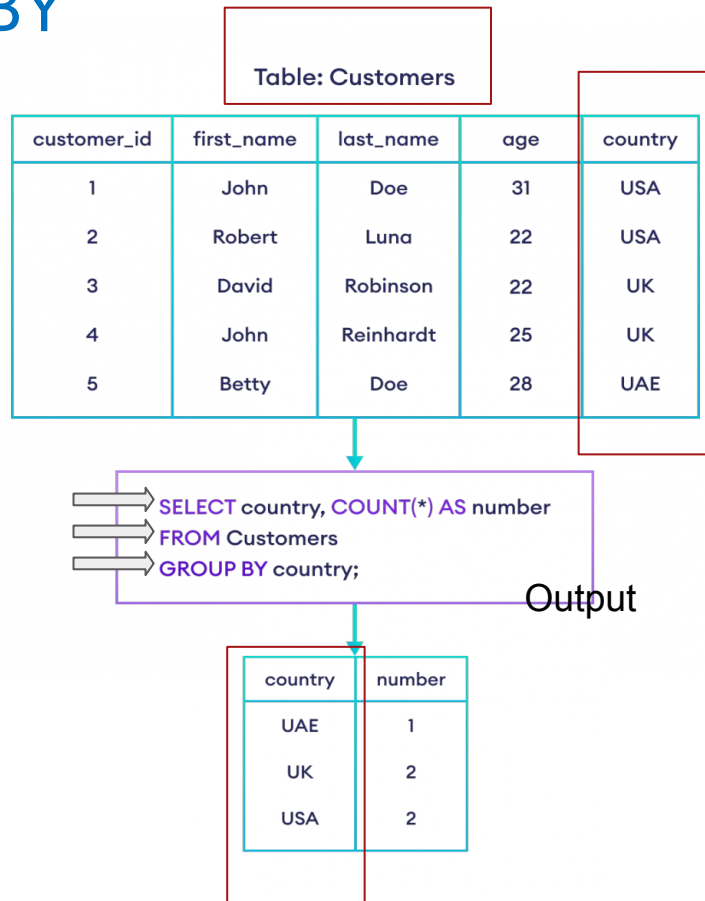
Now, if you want to know the **total sales for each employee**, you would use GROUP BY :

```
SELECT employee_id, SUM(sales_amount) AS total_sales
FROM sales
GROUP BY employee_id;
```

Result:

employee_id	total_sales
1	900
2	1000
3	600

8. How a SELECT query is structured **Aggregating** Data with **GROUP BY**



COUNT(*) AS number: **counts the number of rows for each unique value** in the 'country' column and assigns the result to a new column alias 'number'.

9. How a SELECT query is structured **Aggregating** Data with **GROUP BY** and **HAVING**

HAVING: It is used to filter the groups **after the aggregation** has been performed.

Table: sales

employee_id	sales_amount
1	500
2	300
1	400
2	700
3	600

Write an SQL query to find employees whose total sales exceed 800, considering only sales records with amounts greater than 200;

```
SELECT employee_id,  
SUM(sales_amount) AS total_sales  
FROM sales  
WHERE sales_amount > 200  
GROUP BY employee_id  
HAVING SUM(sales_amount) > 800;
```

Result:

employee_id	total_sales
1	900
2	1000

9. How a SELECT query is structured **Aggregating** Data with **GROUP BY** and **HAVING**

HAVING: It is used to filter the groups **after the aggregation** has been performed.

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

`SELECT COUNT(customer_id), country
FROM Customers
GROUP BY country
HAVING COUNT(customer_id) > 1;`

COUNT(customer_id)	country
2	UK
2	USA

Note: The HAVING clause was introduced because the WHERE clause does not support aggregate functions. Also, GROUP BY must be used before the HAVING clause.

JOINING

DATA

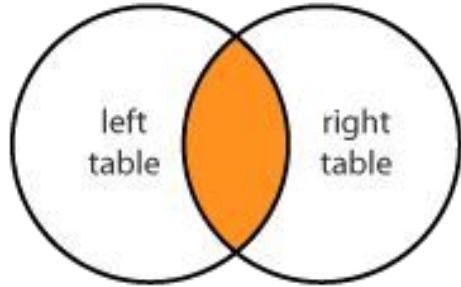
A join operation merges two or more tables into a single relation. Different ways of doing this:

- Inner - Returns only the rows where there is a match in **both** tables.
- Left - Returns all rows from the **left table**, and matching rows from the right table.
- Right- Returns all rows from the **right table**, and matching rows from the left table.
- Full Outer- Returns all rows when there is a match in **either** table.

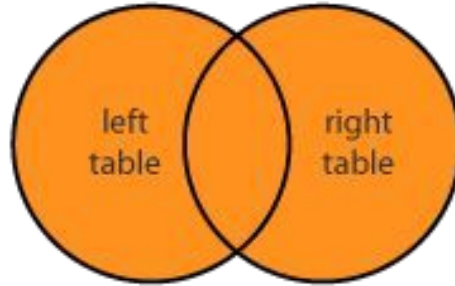
Join operations are done on columns that explicitly link the tables together

SQL JOIN VISUAL

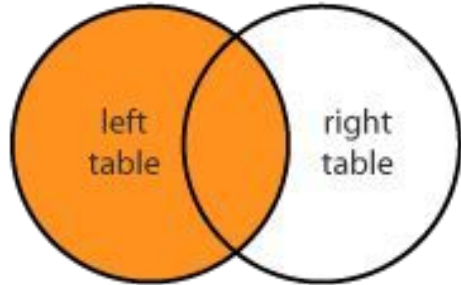
INNER JOIN



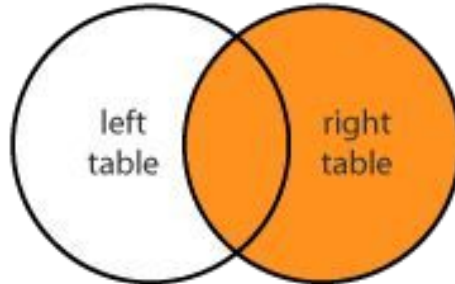
FULL JOIN



LEFT JOIN



RIGHT JOIN



- **INNER JOIN:** Returns only rows with matches in both tables.
- **FULL JOIN:** Returns all rows from both tables, with NULLs where there are no matches.
- **LEFT JOIN:** Returns all rows from the left table, with NULLs for no matches on the right.
- **RIGHT JOIN:** Returns all rows from the right table, with NULLs for no matches on the left.

How a query is structured (INNER JOIN)

Returns only the rows with matching values in both tables.

- INNER join **compares** each row of the first table with each row of the second table, to **find all pairs of rows** that satisfy the join-predicate.
- When the join-predicate is satisfied, the column values from both tables are combined into a new table.

Example: join-predicate :

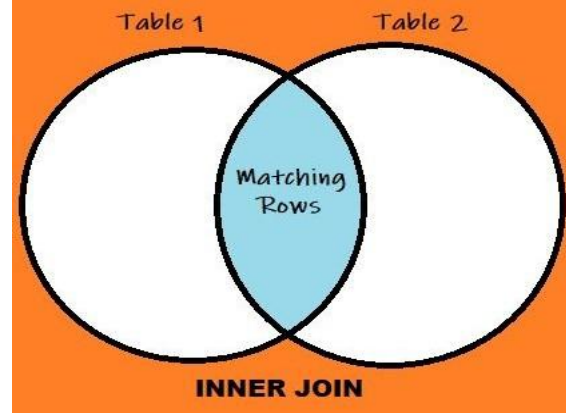
EmpDetails.EmpID = MaritalStatus.EmpID

EmpDetails			MaritalStatus		
ID	Name	Salary	ID	Name	Status
1	John	40000	1	John	Married
2	Alex	25000	3	Simon	Married
3	Simon	43000	4	Stella	Unmarried

+

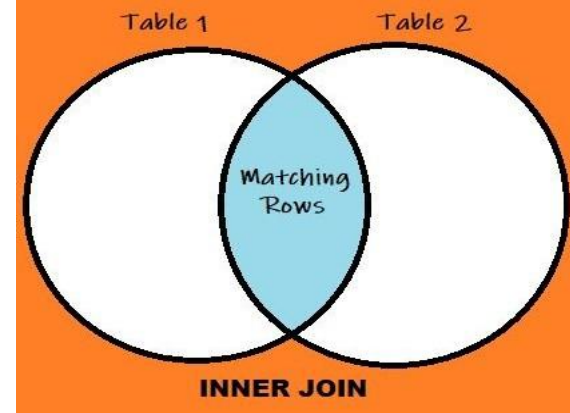
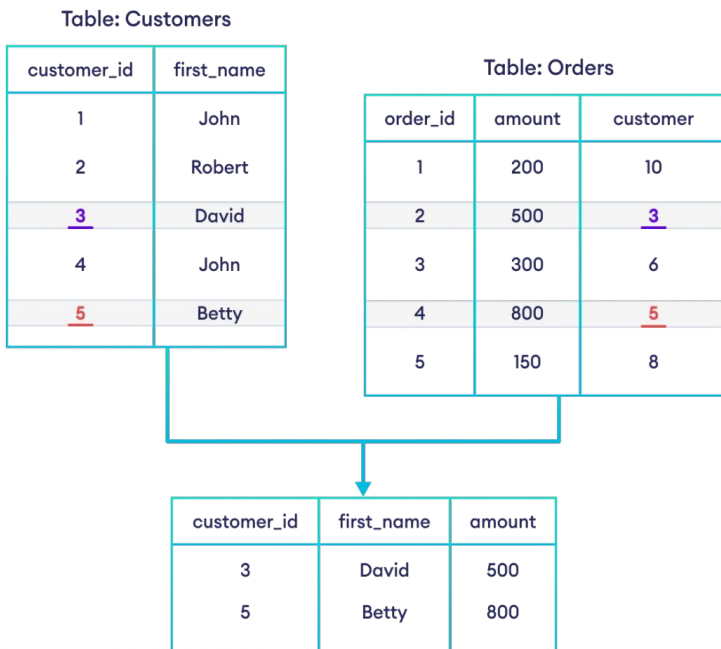
=

ID	Name	Salary	Status
1	John	40000	Married
3	Simon	43000	Married



INNER JOIN selects only **matching left and right table rows**.

How a query is structured (INNER JOIN)



Ex:

```
SELECT Customers.customer_id,  
Customers.first_name, Orders.amount  
FROM Customers  
INNER JOIN Orders  
ON Customers.customer_id =  
Orders.customer;
```

- You can also use * to select all columns
- Use "tablename.columnname" for selecting specific columns

In-Class Exercise

Write equivalent **Pandas** query for given SQLs (dataframe is tips):

```
SELECT *  
FROM tips  
WHERE time = 'Dinner' AND tip > 5.00
```